

INTRODUCTION TO DEEP LEARNING AND PRACTICE

학습 관련 기술 (1) 매개변수 갱신

SGD → Momentum → AdaGrad → RMSProp → Adam

참고: 밑바닥부터 시작하는 딥러닝 6장

이번 주의 핵심 질문

같은 gradient 정보로 더 빠르고 안정적으로 최솟값을 찾을 수는 없을까?

방향의 문제

-gradient가 늘 최솟값을 향하지는 않는다

보폭의 문제

학습률 α 하나로 모든 매개변수를 같은 비율로 움직인다

달라지는 것은 오직 갱신 규칙뿐

기법	갱신 규칙	개선한 점
SGD	$W \leftarrow W - \alpha g$	(기준)
Momentum	$v \leftarrow \beta v - \alpha g, W \leftarrow W + v$	방향: 진동 상쇄, 가속
AdaGrad	$h \leftarrow h + g \odot g, W \leftarrow W - \alpha g / \sqrt{h}$	보폭: 매개변수별 학습률
RMSProp	$h \leftarrow \rho h + (1-\rho) g \odot g$	학습률이 0으로 줄지 않음
Adam	m, v의 지수이동평균 + 편향 보정	Momentum + RMSProp

예제 함수

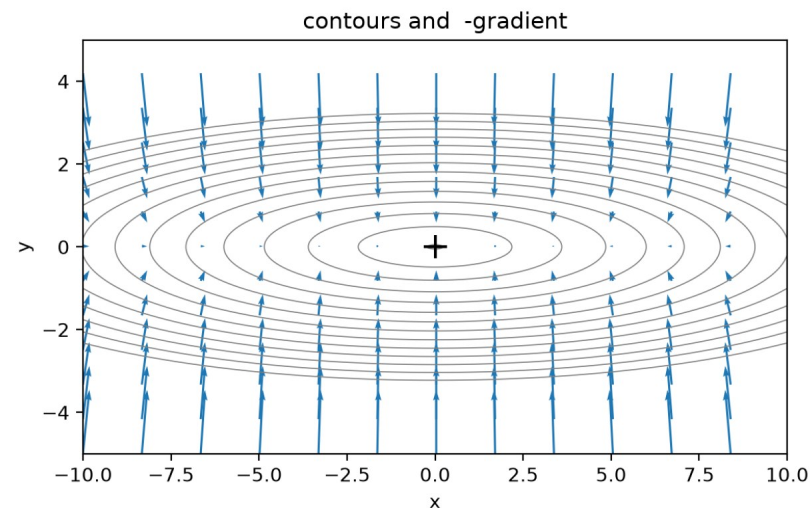
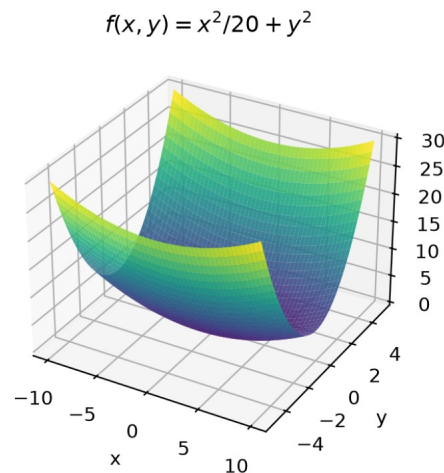
$$f(x, y) = x^2/20 + y^2$$

$$\nabla f(x, y) = (x/10, 2y)$$

최솟값: (0, 0)에서 0

시작점: 모든 기법에서 (-7, 2)

x 방향은 완만, y 방향은 가파른 골짜기



예제 함수

두 방향의 곡률이 20배 차이 난다

x 방향

$$\partial^2 f / \partial x^2 = 1/10$$

아주 완만하다

y 방향

$$\partial^2 f / \partial y^2 = 2$$

가파르다

그래서 대부분의 점에서 $-\nabla f$ 는 최솟값이 아니라 **골짜기의 벽(y축 방향)**을 가리킨다.
실제 손실함수에서도 방향마다 곡률이 크게 다른 경우가 흔하다.

모든 기법은 update(params, grads) 하나로

```
def grad_f(x, y):  
    return x / 10.0, 2.0 * y  
  
def minimize(optimizer, start=(-7.0, 2.0), steps=30):  
    params = {"x": np.array(start[0]), "y": np.array(start[1])}  
    path = [(float(params["x"]), float(params["y"]))]  
    for _ in range(steps):  
        gx, gy = grad_f(params["x"], params["y"])  
        grads = {"x": gx, "y": gy}  
        optimizer.update(params, grads)  
        path.append((float(params["x"]), float(params["y"])))  
    return np.array(path)
```

params, grads는 지난주 mnist_lab.py의 network.params, grads와 같은 형태 → MNIST 학습 루프에도 그대로 쓸 수 있다.

1 · 확률적 경사하강법 (SGD)

지금까지 써 온 방법

$$W_{k+1} = W_k - \alpha g_k$$

확률적: g_k 를 무작위로 고른 미니배치의 평균 손실로 계산하기 때문 (지난주)

```
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    def update(self, params, grads):
        for key in params:
            params[key] -= self.lr * grads[key]
```

좌표마다 따로 보면

$$\mathbf{x}_{k+1} = (1 - \alpha/10) \mathbf{x}_k$$

매 단계 $\mathbf{x}$ 는 $(1 - \alpha/10)$ 배

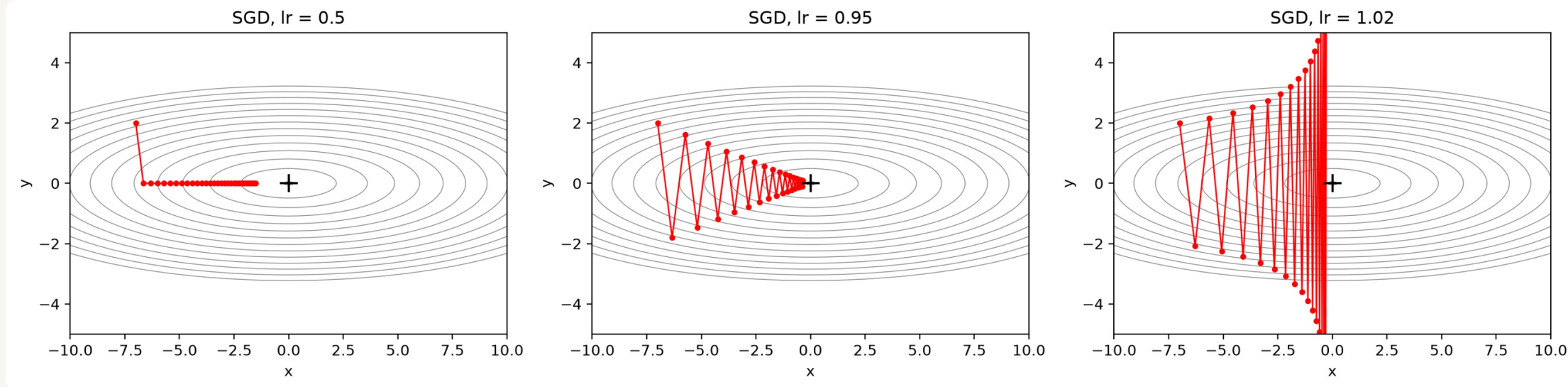
$$\mathbf{y}_{k+1} = (1 - 2\alpha) \mathbf{y}_k$$

매 단계 $\mathbf{y}$ 는 $(1 - 2\alpha)$ 배

- $\mathbf{y}$ 가 발산하지 않으려면 $|1 - 2\alpha| < 1$, 즉 $\alpha < 1$
- 그런데 $\alpha < 1$ 이면 $\mathbf{x}$ 의 비율은 $1 - \alpha/10 > 0.9 \rightarrow \mathbf{x}$ 는 한 단계에 10%도 줄지 않는다
- 가파른 방향 때문에 α 를 키울 수 없고, 그 α 로는 완만한 방향이 느리다

1 · SGD

학습률에 따른 SGD의 경로 (30단계)



$\alpha = 0.5$

y는 한 번에 0, x는 0.95배씩 $\rightarrow$ 30단계 뒤 x ≈ -1.5

$\alpha = 0.95$

y 비율 -0.9 $\rightarrow$ 부호가 바뀌며 **지그재그**

$\alpha = 1.02$

y 비율 $|1 - 2.04| = 1.04 > 1 \rightarrow$ **발산**

SGD의 약점

이후의 기법들은 두 문제를 하나씩 고친다

방향의 문제

이동 방향이 언제나 현재의 $-g \rightarrow$ 곡률이 다르면 최솟값을 향하지 않는다

$\rightarrow$ **Momentum**

보폭의 문제

α 하나가 모든 방향의 보폭을 정한다 $\rightarrow$ 가파른 쪽에 맞추면 완만한 쪽이 느리다

$\rightarrow$ **AdaGrad, RMSProp**

둘을 합치면 $\rightarrow$ **Adam**

2 · 모멘텀 (Momentum)

경사면에 공을 굴린다

공은 그 순간의 경사만 따라가지 않는다. 이미 가진 **속도**가 있어서, 같은 방향으로 점점 빨라지고 자주 바뀌는 방향의 움직임은 상쇄된다.

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k - \alpha \mathbf{g}_k, \quad \mathbf{W}_{k+1} = \mathbf{W}_k + \mathbf{v}_{k+1}$$

$$\mathbf{v}_0 = 0$$

- $\mathbf{v}$: 속도 (velocity)
- $\beta \in [0, 1)$, 보통 0.9: 이전 속도를 얼마나 유지할지 (마찰의 역할)
- $\beta = 0$ 이면 SGD와 같다

2 · 모멘텀

속도는 과거 gradient의 가중 합이다

$$\mathbf{v}_{k+1} = -\alpha (\mathbf{g}_k + \beta \mathbf{g}_{k-1} + \beta^2 \mathbf{g}_{k-2} + \cdots + \beta^k \mathbf{g}_0)$$

부호가 매번 바뀌는 방향 (y)

더한 값이 상쇄 → 진동이 줄어든다

부호가 계속 같은 방향 (x)

더한 값이 누적 → 점점 빨라진다

최대 $\alpha/(1-\beta) = 10\alpha$ ($\beta=0.9$)

손계산 ($\alpha=0.1, \beta=0.9$, 시작 $(-7, 2)$): $\mathbf{v}_1 = (0.07, -0.4) \rightarrow (-6.93, 1.6)$, $\mathbf{v}_2 = (0.1323, -0.68) \rightarrow (-6.7977, 0.92)$

코드: 속도 v 를 기억한다

```
class Momentum:
    def __init__(self, lr=0.01, beta=0.9):
        self.lr = lr
        self.beta = beta
        self.v = None

    def update(self, params, grads):
        if self.v is None:                                # velocity starts at 0
            self.v = {key: np.zeros_like(val) for key, val in params.items()}
        for key in params:
            self.v[key] = self.beta * self.v[key] - self.lr * grads[key]
            params[key] += self.v[key]
```

매개변수마다 학습률을 다르게

$$\mathbf{h}_{k+1} = \mathbf{h}_k + \mathbf{g}_k \odot \mathbf{g}_k, \quad \mathbf{W}_{k+1} = \mathbf{W}_k - \alpha \mathbf{g}_k / (\sqrt{\mathbf{h}_{k+1}} + \epsilon)$$

$\mathbf{h}$: 지금까지의 gradient 제곱의 합 (성분별), $\mathbf{h}_0 = 0$, $\epsilon \approx 10^{-7}$

매개변수 하나의 실제 학습률 = $\alpha / \sqrt{(g_0^2 + g_1^2 + \dots + g_k^2)} \rightarrow$ gradient가 컸던 매개변수일수록 보폭이 작아진다

예제의 첫 단계: $\mathbf{W}_1 = \mathbf{W}_0 - \alpha \mathbf{g}_0 / |\mathbf{g}_0|$

각 좌표가 gradient의 크기와 관계없이 **정확히 α 만큼** 움직인다.

$\alpha = 1.5$, $\mathbf{g}_0 = (-0.7, 4) \rightarrow (\mathbf{x}_1, \mathbf{y}_1) = (-5.5, 0.5)$

코드와 약점

```
class AdaGrad:
    def __init__(self, lr=0.01, eps=1e-7):
        self.lr = lr
        self.eps = eps
        self.h = None

    def update(self, params, grads):
        if self.h is None:
            self.h = {key: np.zeros_like(val) for key, val in params.items()}
        for key in params:
            self.h[key] += grads[key] * grads[key] # sum of squared gradients
            params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + self.eps)
```

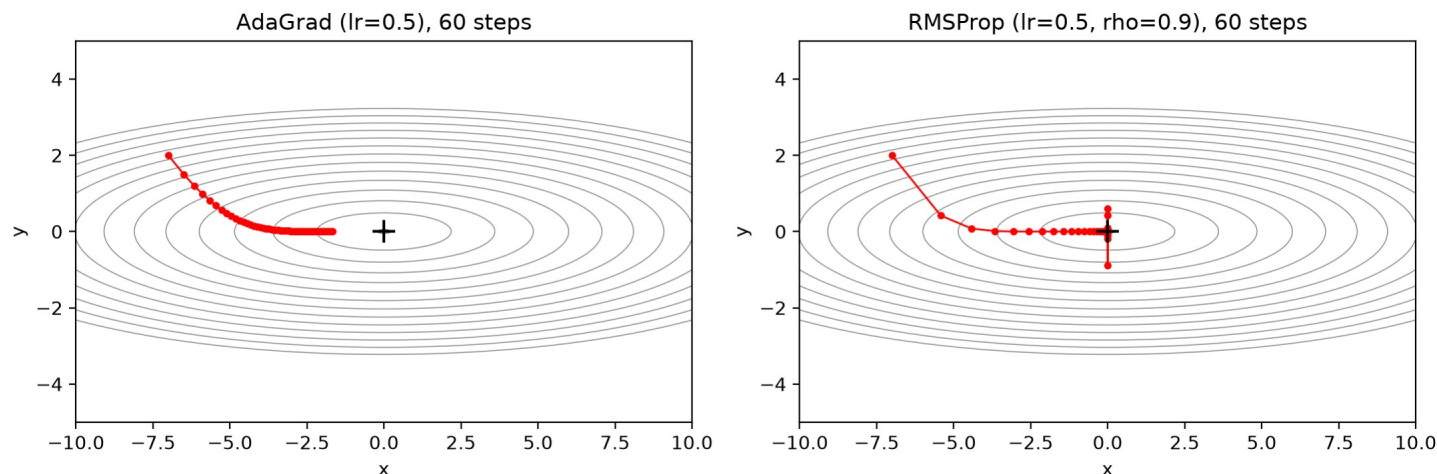
약점: h 는 계속 더해지기만 한다 → 실제 학습률 $\alpha/\sqrt{h}$ 가 계속 작아져 결국 거의 멈춘다

3 · RMSProp

오래된 gradient는 서서히 잊는다

$$\mathbf{h}_{k+1} = \rho \mathbf{h}_k + (1 - \rho) \mathbf{g}_k \odot \mathbf{g}_k, \quad \mathbf{W}_{k+1} = \mathbf{W}_k - \alpha \mathbf{g}_k / (\sqrt{\mathbf{h}_{k+1}} + \epsilon)$$

지수이동평균 ($\rho \approx 0.9$): $\mathbf{h}$ 는 최근 gradient 제공의 **평균** 정도로 유지 $\rightarrow$ 학습률이 0으로 줄지 않는다



같은 $\alpha = 0.5$ 로 60단계: AdaGrad(왼쪽)는 점점 느려져 멈추고, RMSProp(오른쪽)은 최솟값 근처까지 간다

Momentum + RMSProp

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k$$

$$\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k \odot \mathbf{g}_k$$

$$\hat{\mathbf{m}}_k = \mathbf{m}_k / (1 - \beta_1^k), \quad \hat{\mathbf{v}}_k = \mathbf{v}_k / (1 - \beta_2^k)$$

$$\mathbf{W}_k = \mathbf{W}_{k-1} - \alpha \hat{\mathbf{m}}_k / (\sqrt{\hat{\mathbf{v}}_k} + \epsilon)$$

$\mathbf{m}$: gradient의 평균

어느 방향으로 갈까 (1차 moment)

$\mathbf{v}$: gradient 제곱의 평균

보통 얼마나 큰가 (2차 moment)

$\hat{\mathbf{m}}/\sqrt{\hat{\mathbf{v}}}$: 평균 방향을 크기로 나누어 보폭을 맞춘다

보통 $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\mathbf{m}_0 = \mathbf{v}_0 = 0$ ($\mathbf{g}_k$ 는 $\mathbf{W}_{k-1}$ 에서의 gradient)

편향 보정 (bias correction)은 왜 필요한가

m과 v를 0에서 시작하므로 초기에는 실제 평균보다 작게 치우친다.

보정 전

$$m_1 = (1 - 0.9) g_1 = 0.1 g_1$$

실제 gradient의 1/10밖에 안 된다



보정 후

$$\hat{m}_1 = 0.1 g_1 / (1 - 0.9) = g_1$$

k가 커지면 $\beta^k \rightarrow 0$, 보정 효과는 사라진다

예제 첫 단계: $\hat{m}_1 = g_1$, $\hat{v}_1 = g_1 \odot g_1 \rightarrow W_1 = W_0 - \alpha g_1 / |g_1|$, 각 좌표가 α 만큼 이동
 $\alpha = 0.3$ 이면 $(x_1, y_1) = (-6.7, 1.7)$

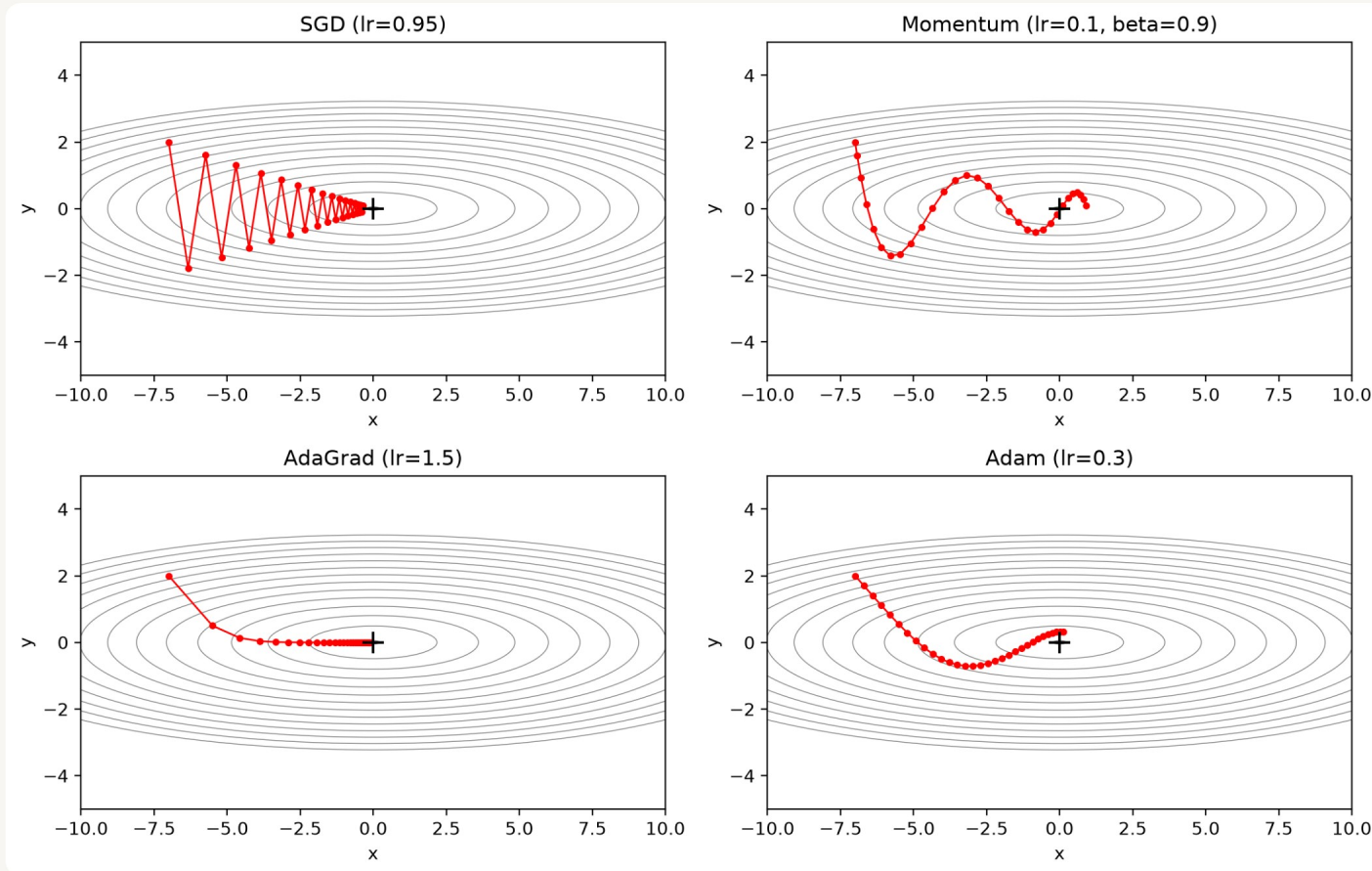
코드: update 메서드

```
def update(self, params, grads):
    if self.m is None:
        self.m = {key: np.zeros_like(val) for key, val in params.items()}
        self.v = {key: np.zeros_like(val) for key, val in params.items()}
    self.k += 1
    for key in params:
        self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) * grads[key]
        self.v[key] = self.beta2 * self.v[key] + (1 - self.beta2) * grads[key] ** 2
        m_hat = self.m[key] / (1 - self.beta1 ** self.k) # bias correction
        v_hat = self.v[key] / (1 - self.beta2 ** self.k)
        params[key] -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
```

`__init__`은 `lr=0.001`, `beta1=0.9`, `beta2=0.999`, `eps=1e-8`과 단계 수 `k = 0`, `m = v = None`을 저장한다 (`optimizers_2d.py`).

5 · 비교

같은 시작점 (-7, 2)에서 30단계



SGD ($\alpha=0.95$)

y로 지그재그, x는 느림

Momentum ($\alpha=0.1, \beta=0.9$)

크게 휘며 가속, 지나쳤다 돌아옴

AdaGrad ($\alpha=1.5$)

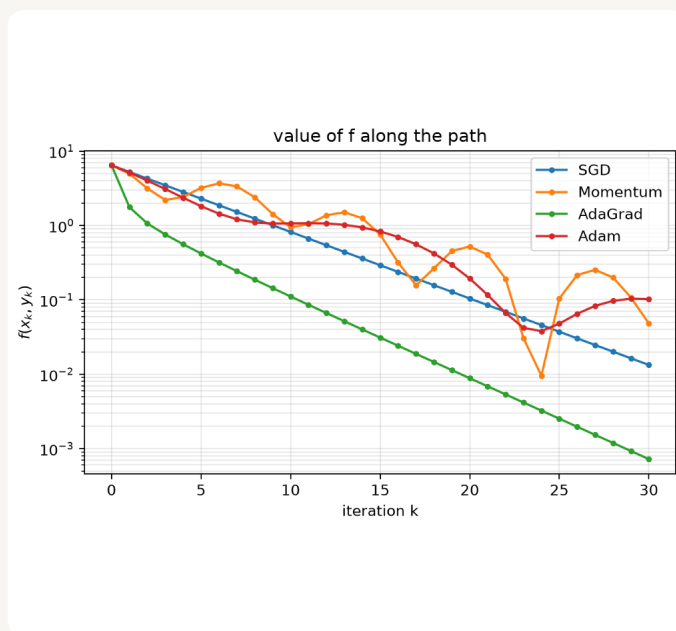
진동을 바로 잡고 바닥을 따라 곧게

Adam ($\alpha=0.3$)

모멘텀처럼 휘지만 진동이 작음

5 · 비교

경로를 따라 계산한 f 값과 첫 두 단계



기법	(x_1, y_1)	(x_2, y_2)
SGD	$(-6.335, -1.8)$	$(-5.733, 1.62)$
Momentum	$(-6.93, 1.6)$	$(-6.798, 0.92)$
AdaGrad	$(-5.5, 0.5)$	$(-4.573, 0.136)$
Adam	$(-6.7, 1.7)$	$(-6.400, 1.402)$

AdaGrad와 Adam의 첫 단계는 두 좌표가 모두 정확히 α 만큼 움직인다.

정리

매개변수 갱신 기법

기법	갱신 규칙	개선한 점
SGD	$W \leftarrow W - \alpha g$	(기준)
Momentum	$v \leftarrow \beta v - \alpha g, W \leftarrow W + v$	방향: 진동 상쇄, 가속
AdaGrad	$h \leftarrow h + g \odot g, W \leftarrow W - \alpha g / \sqrt{h}$	보폭: 매개변수별 학습률
RMSProp	$h \leftarrow \rho h + (1-\rho) g \odot g, W \leftarrow W - \alpha g / \sqrt{h}$	학습률이 0으로 줄지 않음
Adam	m, v의 지수이동평균 + 편향 보정	Momentum + RMSProp

어떤 기법이 가장 좋은가?

이 예제의 순위로 결론을 내릴 수는 없다

- 결과는 함수의 모양, 시작점, 학습률 같은 hyper-parameter에 따라 크게 달라진다
- 실제 딥러닝: Adam은 hyper-parameter에 덜 민감해 기본값으로 많이 쓰인다
- 잘 조정된 SGD (+ 모멘텀)가 최종 성능이 더 좋은 경우도 많다

PyTorch에서는

```
torch.optim.SGD(..., momentum=0.9) · torch.optim.Adagrad  
torch.optim.RMSprop · torch.optim.Adam
```

연습문제

손으로, 그리고 코드로 확인하기

1. $f(x, y) = x^2/20 + y^2$ 에서 SGD가 발산하지 않는 α 의 범위와, 그 안에서 x 가 가장 빨리 수렴하는 α 를 구하라.
2. 모멘텀에서 gradient가 매 단계 g 로 일정할 때 $v_k \rightarrow -\alpha g/(1-\beta)$ 임을 보이라.
3. 시작점 $(-7, 2)$ 에서 AdaGrad ($\alpha = 1.5$)의 두 번째 단계 (x_2, y_2) 를 손으로 계산하라.
4. Adam에서 편향 보정이 없으면 첫 단계의 이동량은 얼마인가? 보정이 왜 필요한가?
5. mnist_lab.py의 갱신 부분을 `optimizer.update(network.params, grads)`로 바꾸어 네 기법을 5 epoch씩 비교하라.

코드: optimizers_2d.py · 강의 노트 연습문제와 같다